Module 1 — Dev Environment & Collaboration

Reading

> ***Role of this reading:*** *This reading is your canonical reference for Module 1. It defines terminology precisely, documents procedures step-by-step, and addresses edge cases you will encounter in the lab and integration task. Read it before the concept session. Return to it as reference material throughout the module.*

Learning Outcomes

By the end of this module, you will be able to:

1. Create and manage a reproducible Python environment using venv and `requirements.txt` as the cohort standard, and explain when teams choose conda instead.
2. Apply a Git feature-branch workflow: create branches, write meaningful commit messages, push to a remote, open pull requests, and interpret and resolve basic merge conflicts.
3. Configure JupyterLab and VS Code for data science workflows, including linking a venv to a JupyterLab kernel and using relevant extensions.
4. Evaluate AI-assisted coding tools — including agentic IDEs, completion tools, and CLI agents — critically: understanding their amplifier role, their limitations, and when to apply human judgment before accepting a suggestion.
5. Structure a collaborative project repository with professional conventions: complete README, `.gitignore`, `requirements.txt`, reproducibility script (`setup.sh` or Makefile), and an `AGENTS.md` governance file.

Module Structure

This module runs across five days. Here is how the pieces connect:

Stage	When	Purpose
This reading	Before concept session (Day 1)	Canonical reference — defines terminology, procedures, and edge cases for everything in the module
Concept session	Day 1 evening	Builds the conceptual schema for *why* environments and branches work the way they do
Core Skills Drill	Day 1 evening (after the concept session)	Confirms you can execute the key procedures from memory in a project context
Applied Lab	Days 2–3	Applies venv and Git workflows to a realistic project scenario with automated grading
Integration Task	Days 4–5	Extends the lab into a team-ready project artifact — documentation, automation, and governance

What success looks like: By the end of this module, you should be able to clone a repo, create and activate a virtual environment, install dependencies, confirm the environment works, and open a pull request from a feature branch — without referring to a guide.

In This Reading

1. The Reproducibility Problem in Applied Data Science
2. Python Environment Management
3. Git Branching Workflows
4. Docker: Conceptual Introduction
5. JupyterLab Configuration
6. VS Code Configuration for Data Science
7. AI-Assisted Coding: Capabilities and Critical Use
8. Repository Structure Conventions

- Environment Setup Troubleshooting
- Key Terms
- Summary
- What's Next

> ***How to use this reading:*** *Sections 2, 3, and 8 are the most directly applied in the drill, lab, and integration task. Sections 4 (Docker), 5 (JupyterLab), 6 (VS Code), and 7 (AI tools) provide context for the tools you will work in throughout the course. All sections are reference material — return to any of them as needed.*

---

1. The Reproducibility Problem in Applied Data Science

1.1 "Works on My Machine" as a Professional Liability

Imagine you finish a data analysis script, push it to GitHub, and a teammate clones it and runs it. Instead of results, they get:

```
ModuleNotFoundError: No module named 'pandas'
```

Or worse — it runs, but produces different numbers because a library updated a function's default behavior between versions. You both run "the same code." You get different answers.

This is the **reproducibility problem**, and it is one of the most common sources of friction in collaborative data science work. It is not an exotic edge case — it happens constantly in professional teams and in research. The "works on my machine" failure mode is a professional liability because it shifts debugging burden onto your collaborators and erodes trust in your outputs.

Reproducibility in this context means: given the same code, the same environment, and the same data, anyone running the code should get the same result. A **reproducible environment** is one that can be reconstructed from a specification file, on any compatible machine, without ambiguity.

1.2 Why Package Versions Matter

Python libraries are not static. Between versions, APIs change, defaults shift, and functions are deprecated or removed. Consider:

- `pandas` changed how groupby aggregation handles certain edge cases between 1.x and 2.x
- `scipy` changed default parameter values for several optimization routines in 1.8.x, which can silently change numerical results
- `matplotlib` changed its default color cycle in version 2.0

If you install "whatever the latest version is" today, and your teammate installs "whatever the latest version is" three months from now, you may be running meaningfully different code under the hood — with no visible warning.

A **`requirements.txt`** file is how you make the environment specification explicit and portable. It is an engineering contract: "if you install exactly these packages at these versions, this code will behave as I developed it."

1.3 Deterministic vs. Non-Deterministic Code

Deterministic code produces the same output every time given the same input — the environment, the algorithm, and the data all produce one predictable result. **Non-deterministic** code may produce different outputs even with identical inputs: training a neural network with a random seed unset, for example, or any function that reads from a live data source.

Environment reproducibility is one necessary layer of determinism — it controls one source of variation. Other layers (random seeds, data versioning) come later in the course. For now, the rule is: if your environment is not specified, you've introduced a hidden variable into every result you produce.

---

2. Python Environment Management

2.1 venv: The Cohort Standard

venv is Python's built-in virtual environment module. It ships with Python 3.3 and later — no separate installation needed. A virtual environment is an isolated Python installation with its own packages, separate from your system Python and from every other project you work on.

Why isolated environments? Without them, all projects on your machine share a single Python installation. Installing a package for Project A might break Project B if the versions conflict. Environments prevent this by giving each project its own dependency tree.

Creating a venv

```
python -m venv .venv
```

This creates a `.venv/` directory in your current folder containing a complete Python installation. The convention is `.venv` (dot-prefixed so it's hidden on Unix systems and clearly a tool directory, not source code). **Always add `.venv/` to your `.gitignore` — never commit the venv directory to Git.**

Activating a venv

Activation sets your terminal session to use the venv's Python and pip instead of the system versions. **You must activate your venv in every new terminal session.** The activation command differs by platform:

OS	Shell	Command
Mac / Linux	bash or zsh	`source .venv/bin/activate`
Windows	Git Bash	`source .venv/Scripts/activate`
Windows	Command Prompt	`.venv\Scripts\activate.bat`
Windows	PowerShell	`.venv\Scripts\Activate.ps1`

Confirming activation: After running the activation command, your terminal prompt changes to show `(.venv)` as a prefix:

```
(.venv) user@machine:~/project$
```

If you don't see this prefix, the venv is not active and pip installs will go to the wrong place.

Windows PowerShell note: If running `Activate.ps1` gives you a security error about scripts being disabled, run this once in your PowerShell session to allow it:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Installing from requirements.txt

```
pip install -r requirements.txt
```

This reads the `requirements.txt` file and installs every listed package. Run this immediately after activating your venv and cloning a repo.

Freezing your dependencies

Once your environment is working, capture the exact versions of every installed package:

```
pip freeze > requirements.txt
```

This overwrites `requirements.txt` with pinned versions (e.g., `pandas==2.1.4`). **Do this before committing if you want exact reproducibility.** For shared course projects, pinned versions are preferred.

Rebuilding from scratch

If your environment gets corrupted or you need a clean start:

```
# Deactivate first
deactivate

# Delete the existing venv
rm -rf .venv          # Mac/Linux
rd /s /q .venv        # Windows CMD

# Recreate and reinstall
python -m venv .venv
source .venv/bin/activate    # or the Windows equivalent
pip install -r requirements.txt
```

This is always cleaner than trying to patch a broken environment. A well-written `requirements.txt` makes rebuilding trivial.

Common pip issues

- **Outdated pip:** If you see warnings about pip being old, upgrade it first: `pip install --upgrade pip`
- **Version conflicts:** If two packages require incompatible versions of a shared dependency, pip will report the conflict. In Tier 1 (Modules 1–4), this is rare because the requirement set is pure-Python and carefully chosen.
- **"Not found" for a package:** Check spelling. Package names are case-insensitive in pip but lowercase by convention. `Pandas` and `pandas` install the same thing, but the test autograder checks lowercase.

2.2 conda: Conceptual Overview

conda is a cross-platform package and environment manager maintained by Anaconda. Unlike venv, conda can manage non-Python packages, binary dependencies, and even different Python versions within the same installation. It uses its own package registry (the `conda-forge` channel is the most commonly used) in addition to PyPI.

This course uses venv as its standard. You will not create conda environments for any lab or integration task. Understanding conda's purpose prepares you for team environments that use it.

When do professional teams choose conda?

- **C-extension packages:** Some scientific computing packages (like certain compiled linear algebra libraries) have binary components that pip can't reliably build on all platforms. conda distributes pre-compiled binaries.
- **CUDA and GPU dependencies:** PyTorch GPU builds, CUDA toolkit, cuDNN — managing GPU software stacks is much easier with conda than pip.
- **Cross-language projects:** conda manages R packages and other non-Python dependencies alongside Python packages in a single environment file.
- **Large team standardization:** Organizations that need to lock down exact binary versions across dozens of machines often prefer conda's stricter reproducibility guarantees.

conda's environment specification file is `environment.yml` :

```
name: myproject
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.11
  - pandas=2.1
  - matplotlib=3.8
  - pip:
    - some-pypi-only-package
```

The `pip:` block lets you mix conda packages with pip-only packages in the same environment file.

When you encounter conda on a team: If a project provides an `environment.yml` , use `conda env create -f environment.yml` to recreate the environment, then `conda activate <name>` to use it. If you're working with a Windows-only install issue that venv can't resolve, the install plan for this course documents conda as a fallback path. In all other situations, use venv.

---

3. Git Branching Workflows

3.1 Review: The Git Mental Model

You completed basic Git in Pre-Work: `git init` , `git add` , `git commit` , `git push` , `git pull` . The mental model: Git tracks a **commit graph** — a directed acyclic graph of snapshots. Each commit records a complete state of your tracked files, linked to its parent commit.

A **remote** is a reference to a Git repository hosted elsewhere (on GitHub, in this course). **HTTPS + PAT** is your authentication method: when Git prompts for a password, you provide a GitHub Personal Access Token, not your account password.

This module extends that foundation to **branching** — the practice of maintaining parallel lines of work in the same repository.

3.2 Why main Is Protected

`main` is the default primary branch in a Git repository. On professional teams, `main` (or sometimes `master` in older repos) represents the stable, deployable, trustworthy state of the project. Team members don't push directly to `main` — they work on feature branches and merge their work into `main` only after it's been reviewed.

Why protect main?

- Pushing directly to main with a bug affects everyone who pulls from it immediately
- There's no review step — errors land without a second set of eyes
- It's harder to identify when a bug was introduced (no PR trail)
- CI/CD pipelines often deploy from main automatically — a bad push can break production

The feature branch model solves this by making the review step mandatory.

3.3 Feature Branches: Naming and Lifecycle

A **branch** is a movable pointer to a commit — a named line of development. Creating a branch is instantaneous and costs nothing in Git; it's just a pointer that moves forward as you commit.

Branch naming conventions:

Prefix	Use case	Example
`feature/`	New functionality	`feature/add-data-pipeline`
`setup/`	Environment or infrastructure changes	`setup/environment`
`fix/`	Bug fixes	`fix/null-handling`
`integration/`	Integration task work	`integration/collab-setup`
`drill/`	Drill submissions	`drill/jsmith`

Rules: no spaces (use `-` or `/`); be descriptive; avoid names like `test1` or `my-branch` .

Creating and switching to a branch in one step:

```
git checkout -b feature/my-feature
```

This creates the branch AND switches to it. Equivalent to:

```
git branch feature/my-feature
git checkout feature/my-feature
```

Branch lifecycle:

1. Create a branch from `main` (or the current branch)
2. Make commits on your branch
3. Push your branch to the remote
4. Open a pull request (PR) to merge into `main`
5. PR is reviewed, CI checks pass, then merged
6. Delete the feature branch (optional cleanup)

3.4 Commit Message Conventions

Commit messages are documentation. A well-written commit history tells a story: what changed, when, and why. The standard convention:

- **Subject line:** 50 characters or less; imperative mood ("Add data validation" not "Added data validation"); no period at end
- **Body (optional):** After a blank line; explain *why* the change was made, not just what; wrap at 72 characters

```
Add hospital admission data schema validation

The upstream CSV sometimes omits the admission_date column.
Without schema validation, the pipeline silently produces
wrong aggregation results. This adds a check that exits
early with a descriptive error if required columns are missing.
```

Bad commit messages:

```
fix
updated stuff
wip
asdf
```

These are useless to your future self and useless to a reviewer.

3.5 Pushing a New Branch

When you push a branch to GitHub for the first time, you need to set the upstream tracking reference:

```
git push --set-upstream origin feature/my-feature
```

Or the shorthand:

```
git push -u origin feature/my-feature
```

After the first push, subsequent pushes on that branch just need `git push` .

3.6 Opening a Pull Request

A **pull request (PR)** is a proposal to merge one branch into another, with a review and discussion interface. On GitHub:

1. After pushing your branch, GitHub will show a "Compare & pull request" banner — click it
2. Or go to the repository → "Pull requests" → "New pull request"
3. Set the **base branch** (what you're merging into, usually `main`) and the **compare branch** (your feature branch)
4. Write a **title** (concise, imperative mood, what the PR does)
5. Write a **description** (what changed, why, how to test it — see the PR description template in the integration guide for this module's format)
6. Assign a reviewer if applicable
7. Click "Create pull request"

A well-written PR description includes:

- What changed (bullet list of files added/modified)
- Why each change was made
- How a reviewer should test it (exact commands)

3.7 Reviewing a PR

When reviewing someone else's PR:

- Read the diff: changed lines highlighted in green (additions) and red (deletions)
- Leave inline comments on specific lines by clicking the `+` icon
- Use "Request changes" if something must be addressed before merging
- Use "Approve" when the PR is ready to merge
- Leave a general comment if you have questions that aren't blocking

3.8 Merging Strategies

GitHub offers three merge strategies:

Strategy	What it does	When to use
Merge commit	Creates a new commit that joins both branches; preserves the full branch history	Default; shows the branch in the commit graph
Squash and merge	Combines all branch commits into one commit on main	Keeps main history clean; good for small feature branches
Rebase and merge	Replays branch commits on top of main without a merge commit	Keeps linear history; requires clean commits

For this course: squash and merge is generally preferred for lab and integration submissions — it keeps the main branch history readable.

3.9 Merge Conflicts

A **merge conflict** arises when two branches modify the same lines of the same file, and Git cannot determine which version is correct. Git does not guess — it inserts **conflict markers** into the file and requires you to resolve them manually.

Conflict markers look like this:

```
<<<<<<< HEAD
regional_health_authority = "Ministry of Health"
=======
regional_health_authority = "National Health Authority"
>>>>>>> feature/update-org-names
```

`<<<<<<< HEAD` marks the start of your current branch's version. `=======` separates the two versions. `>>>>>>> feature/update-org-names` marks the end of the incoming branch's version.

To resolve a conflict:

1. Open the conflicted file in VS Code (it highlights conflict regions with a visual editor)
2. Choose which version to keep, or write a new version that incorporates both changes
3. Delete all three conflict marker lines (`<<<<<<<` , `=======` , `>>>>>>>`)
4. Save the file
5. `git add <resolved-file>`
6. `git commit` (Git pre-fills a merge commit message)

VS Code conflict UI: VS Code displays "Accept Current Change," "Accept Incoming Change," "Accept Both Changes," and "Compare Changes" buttons directly above each conflict region. For simple conflicts, click the appropriate button rather than editing manually.

Preventing conflicts: Pull from main before starting work (`git pull origin main`), keep feature branches short-lived, and communicate with teammates about who is editing which files.

3.10 git stash

If you have uncommitted changes and need to switch branches, Git will either refuse (if there's a conflict risk) or carry your changes with you (if they don't conflict). To cleanly stash your current working state:

```
git stash
```

Then switch branches. When you return:

```
git stash pop
```

This restores your changes. `git stash list` shows all stashed states.

3.11 Pre-Commit Hooks

A **pre-commit hook** is a script that runs automatically before a commit is finalized. If the script exits with a non-zero code, the commit is blocked. Teams use hooks to enforce:

- Code formatting (Black, Prettier)
- Linting (flake8, pylint)
- Tests passing
- No secrets committed (truffleHog, detect-secrets)

Hooks live in `.git/hooks/` or are managed by tools like `pre-commit` (the Python package). You won't write hooks in this module, but you'll encounter them in later modules and in professional repos. If a `git commit` fails with an unexpected error message, check whether a pre-commit hook is blocking it.

---

4. Docker: Conceptual Introduction

You installed Docker Desktop and pulled `postgres:15-alpine` in Pre-Work Day 9. This section deepens the mental model you'll need when Docker services become central to the course in Module 3 and beyond.

4.1 Containers vs. Virtual Machines

A **virtual machine (VM)** emulates a complete computer, including its own OS kernel. VMs are heavyweight: they take minutes to start and require gigabytes of RAM per instance.

A **container** is a lightweight, isolated process that shares the host OS kernel but has its own filesystem, networking, and process namespace. Containers start in seconds, consume far less memory, and are portable across any system with Docker installed.

Think of it this way: a VM is like renting an entire apartment; a container is like renting a room in a house with shared infrastructure.

4.2 Key Components

Component	What it is
Dockerfile	A script of instructions for building a Docker image (like a recipe)
Docker image	A read-only snapshot built from a Dockerfile; the "template" for containers
Container	A running instance of an image; isolated, ephemeral, reproducible
Docker Hub	The default public registry for sharing images (like PyPI for containers)
Docker Compose	A tool for defining and running multi-container applications via a `compose.yml` file

4.3 Role of Docker in This Course

- **Pre-Work Day 9:** You pulled `postgres:15-alpine` and confirmed Docker is running. You ran `docker run hello-world` .
- **Module 1:** No Docker commands required in any assignment. This section establishes the conceptual foundation.
- **Module 3:** You will run a relational database service via Docker and connect to it with Python.
- **Module 10:** You will Dockerize a Python API service and run a multi-container stack with Docker Compose.

Docker's role in data science and ML engineering is container-based reproducibility at the infrastructure level: the same way `requirements.txt` guarantees Python package versions, a Docker image guarantees the entire runtime environment — OS, system libraries, and all.

---

5. JupyterLab Configuration

5.1 JupyterLab vs. Classic Jupyter Notebook

JupyterLab is the current-generation interface for Jupyter. It replaced the classic Jupyter Notebook (the single-document interface) and adds:

- A file browser, terminal, text editor, and notebook tabs in one IDE-style layout
- An extension system for adding functionality
- Multiple notebooks open simultaneously in split panes
- Better support for non-notebook files (Markdown, CSV, JSON)

The classic Notebook interface (`jupyter notebook`) still works, but JupyterLab is the standard for new workflows.

5.2 Kernels

A **kernel** is the computational engine that executes the code in a notebook. Each kernel is tied to a specific Python environment. If you run a notebook with the wrong kernel, `import pandas` will fail if pandas isn't installed in that kernel's environment.

The most important JupyterLab habit: Always verify your kernel selection. Look at the top-right corner of a notebook — it shows the active kernel name. If you see "Python 3 (ipykernel)" but you're working in a project-specific venv, your imports may fail or silently use wrong package versions.

5.3 Registering a venv as a JupyterLab Kernel

With your venv activated:

```
python -m ipykernel install --user --name .venv --display-name "aispire"
```

- `--name .venv` : the internal kernel name (no spaces)
- `--display-name "aispire"` : what you'll see in the JupyterLab kernel selector

After running this command, restart JupyterLab. Your "aispire" kernel will appear in the kernel picker. Select it when opening any notebook in this course.

To verify the kernel is using your venv's Python:

```
# In a notebook cell
import sys
print(sys.executable)
# Should print a path ending in .venv/bin/python (or .venv\Scripts\python.exe on
```

5.4 Key JupyterLab Extensions for Data Science

Install extensions through the Extension Manager (puzzle piece icon) or via pip:

Extension	What it adds
`jupyterlab-git`	Git panel: stage, commit, diff from within JupyterLab
`jupyterlab_variableinspector`	Variable inspector panel: view all variables in memory
`jupyterlab-lsp`	Language server: autocomplete, inline error highlighting

5.5 Magic Commands

Magic commands are Jupyter-specific directives prefixed with `%` (line magic) or `%%` (cell magic) that control notebook behavior. They are not Python — they're processed by the IPython kernel.

Magic	What it does
`%timeit <expression>`	Time a single expression (runs it multiple times, reports average)
`%%timeit`	Time an entire cell
`%matplotlib inline`	Render matplotlib plots inline in the notebook (not in a separate window)
`%%bash`	Run the entire cell as a bash script
`%env VAR=value`	Set an environment variable for the current kernel session
`%load_ext autoreload` + `%autoreload 2`	Automatically reload modified modules without restarting the kernel

5.6 When to Use Notebooks vs. Scripts

Use notebooks for	Use scripts for
Exploratory data analysis	Production pipelines
Visualization and storytelling	Automated tasks (cron, CI)
Teaching and documentation	Code that others import as a module
Step-by-step hypothesis testing	Anything that needs `if __name__ == "__main__":`

A common professional pattern: explore in a notebook, then refactor the validated logic into a script in `src/` before integrating it into a pipeline. In this course, you will use both — lab assignments often start with exploration and produce script deliverables.

---

6. VS Code Configuration for Data Science

6.1 Python Extension

The **Python extension** (by Microsoft) is essential. After installation:

- Open the Command Palette (`Ctrl+Shift+P` / `Cmd+Shift+P`) and type "Python: Select Interpreter"
- Choose the interpreter path ending in `.venv/bin/python` (or `.venv\Scripts\python.exe` on Windows)
- This sets VS Code to use your project venv for linting, IntelliSense, and the integrated terminal

6.2 Jupyter Integration in VS Code

VS Code can open and run `.ipynb` notebook files natively with the Jupyter extension. Kernel selection works the same way — choose the "aispire" kernel you registered in Section 5.3.

The VS Code Jupyter interface is slightly different from JupyterLab: notebook cells have inline run buttons, and you can run individual cells or the entire notebook from the toolbar.

6.3 Key Extensions

Extension	Purpose
Python (Microsoft)	Core: interpreter selection, linting, debugging
Pylance (Microsoft)	Fast IntelliSense and type checking
Jupyter (Microsoft)	Native notebook support
GitLens	Enhanced Git annotations: blame, history, PR context inline
Black Formatter	Auto-format Python code to Black style on save

6.4 Settings to Configure

In VS Code's `settings.json` (open via Command Palette → "Preferences: Open User Settings (JSON)"):

```
{
  "editor.formatOnSave": true,
  "[python]": {
    "editor.defaultFormatter": "ms-python.black-formatter"
  },
  "python.linting.enabled": true,
  "python.linting.pylintEnabled": false,
  "python.linting.flake8Enabled": true
}
```

`editor.formatOnSave` runs Black automatically every time you save a Python file, keeping your code formatted without manual effort.

6.5 Terminal Integration

VS Code has an integrated terminal (`Ctrl+`` / `Cmd+``). When you open a terminal in VS Code, it does **not** automatically activate your venv. You must activate it manually:

```
source .venv/bin/activate    # Mac/Linux
.venv\Scripts\activate.bat   # Windows CMD
```

Alternatively, configure VS Code to activate the venv automatically by setting:

```
"terminal.integrated.env.linux": {
    "VIRTUAL_ENV": "${workspaceFolder}/.venv"
}
```

Or simply activate in every terminal session — it takes two seconds and confirms the venv is correct.

6.6 Remote Development (Future Reference)

VS Code's **Remote Development** extensions (Remote - SSH, Remote - Containers, Dev Containers) allow you to develop inside a Docker container or on a remote server as if it were local. You won't use this in Module 1, but it becomes relevant in Modules 10–12 when you deploy services.

---

7. AI-Assisted Coding: Capabilities and Critical Use

7.1 Tool Access in This Cohort

Approximately 20% of learners in this cohort have paid AI tool subscriptions. **80% do not.** All reading content in this section is conceptual — no deliverable in this course requires a paid AI tool or API subscription. Where tool access is relevant to a lab or integration task, free options are always available.

7.2 Types of AI Coding Tools

Free or free-tier available:

Tool	Type	What it offers
GitHub Copilot (free tier)	Completion	Inline code suggestions in VS Code; limited monthly requests on free tier
Claude.ai (free tier)	Chat + code review	Conversation-based coding help, code review, explanation
ChatGPT (free tier)	Chat + code generation	General code generation and explanation
Codeium (free)	Completion	Unlimited free completions in VS Code and other editors
Cursor (free tier)	Agentic IDE	Multi-file edits, codebase chat; limited monthly fast requests on free tier
Ollama (local, fully free)	Local model inference	Run models like Code Llama or Mistral on your own machine — no API, no internet required

Paid / pro tiers: GitHub Copilot Pro, ChatGPT Plus, Claude Pro, Cursor Pro — all offer higher limits, faster models, and additional features.

CLI agents (require API access):

- **Claude Code** — Anthropic's terminal-based coding agent; works on multi-file changes via conversation
- **Codex CLI** — OpenAI's terminal agent
- **Aider** — open-source CLI agent; works with local models via Ollama (free) or commercial APIs

CLI agents differ from completion tools: they don't just suggest the next line — they read your entire codebase, propose multi-file changes, run tests, and iterate. They are powerful and require careful use.

7.3 What These Tools Do Well

- **Boilerplate generation:** Standard patterns (argument parsers, file readers, test stubs, class scaffolding) that are tedious to type and easy to verify
- **Documentation lookup:** "What are the parameters for `pd.read_csv` ?" — faster than searching docs for simple API questions
- **Code explanation:** Pasting unfamiliar code and asking "what does this do?" — useful for navigating existing codebases
- **Test scaffolding:** Generating the structure of a test suite that you then fill with meaningful assertions
- **Refactoring assistance:** Moving functions around, renaming, restructuring — tedious changes where the AI does the mechanical work

7.4 What These Tools Do Poorly

- **Domain-specific correctness:** The tool may generate statistically valid-looking code that produces the wrong answer for your specific domain or data type
- **Security in sensitive contexts:** AI-generated authentication code, SQL queries, and data handling code should be reviewed especially carefully
- **Hallucinated library APIs:** The model may confidently call a function that does not exist, or use a real function with wrong arguments. This is called **hallucination** — not a lie, but a confident confabulation. Always verify AI-suggested function names by checking the documentation or running the code.
- **Unverified architectural changes:** Accepting a large refactoring suggestion without reading it can introduce subtle bugs that are difficult to trace

7.5 The Professional Judgment Framework

Four rules for AI-assisted development:

1. **Review before accepting.** Read every suggestion before pressing Tab or Accept. If you can't explain what the suggested code does, don't accept it.
2. **Test before committing.** Run AI-generated code locally and verify it does what you expect. A passing test is the standard of acceptance, not "it looks right."
3. **Never paste credentials or sensitive data into prompts.** API keys, database passwords, and personal data should never appear in a prompt sent to any external AI service.
4. **Require reproducible outputs.** AI-assisted code must run locally and produce verifiable results. "The AI generated it" is not an explanation of how it works.

7.6 AI as Amplifier

The most important mental model for AI-assisted development: **the tool amplifies whatever skill you bring to it.**

A developer who understands the domain, can read the code, and knows how to test it will use an AI tool to go faster while catching errors. A developer who doesn't understand what they're building will use the tool to produce plausible-looking code that fails in ways they can't diagnose.

This is why Module 1 focuses on fundamentals: environments, Git, project structure. These are the skills the AI tool operates on. Mastering them first means you can evaluate what the tool produces.

7.7 Verification-First Workflow and Human-in-the-Loop (HITL)

Verification-first workflow: Always run code locally and confirm it works before committing it, regardless of where the code came from (AI or your own brain).

Human-in-the-loop (HITL): In this course, all AI-assisted changes require local verification and a passing test before PR submission. You are responsible for every line of code in your submission. "The AI wrote it" is not an answer when a TA asks you to explain your code.

---

8. Repository Structure Conventions

A repository that a teammate can pick up and run in under two minutes is a professional-grade artifact. This section covers the conventions that make that possible.

8.1 Standard Data Science Repository Structure

```
project-name/
├── README.md              ← Required: setup, overview, project structure
├── CHANGELOG.md           ← Chronological record of notable changes
├── AGENTS.md              ← AI contribution governance (see 8.4)
├── .gitignore             ← Files and patterns Git should not track
├── requirements.txt       ← Python dependencies
├── setup.sh               ← Idempotent environment setup script
├── test_environment.py    ← Validates the environment works
├── src/                   ← Production source code (importable modules)
│   └── ...
├── notebooks/             ← Exploratory notebooks (not production code)
│   └── ...
├── data/                  ← Data directory (raw data NOT committed to Git)
│   ├── raw/               ← Original unmodified data
│   └── processed/         ← Cleaned/transformed data
├── tests/                 ← Automated tests
│   └── ...
└── docs/                  ← Documentation
    └── ...
```

Key separation: `src/` is for code that you import and that goes into production. `notebooks/` is for exploration, prototyping, and communication. Code in `notebooks/` is not importable by other scripts. When analysis results from a notebook need to be reproducible and used downstream, refactor the logic into `src/` .

8.2 README.md

A README is the front door of your repository. A complete README for a data science project includes:

1. **Project Title** — the name of the project
2. **Team Members** — names and (optionally) roles
3. **Project Overview** — 2–3 sentences: what problem is being solved and for whom
4. **Data Sources** — what data the project uses; where it comes from; how to obtain it (note if data is not in the repo)
5. **Setup Instructions** — exact commands, step by step, including venv creation, activation (platform-specific), and `pip install -r requirements.txt` ; include a command to verify the setup works (e.g., `python test_environment.py`)
6. **Project Structure** — a directory tree with a one-line explanation of each folder's purpose
7. **Contributing** — branch naming convention, PR process, commit message format

Write the Setup Instructions section as if explaining to someone who has never seen the project before. Use exact commands. If activation differs by OS, include the table.

8.3 .gitignore

A **`.gitignore`** file tells Git which files to ignore — to never track, never stage, never commit. Git will not show ignored files in `git status` . Essential patterns for a Python data science project:

```
# Python
__pycache__/
*.pyc
*.pyo
*.pyd
.Python

# Virtual environment
.venv/
venv/
env/

# Jupyter
.ipynb_checkpoints/

# Data files (never commit raw data to Git)
*.csv
*.parquet
*.xlsx
*.json
data/raw/

# Secrets and credentials
.env
*.key
*.pem
*.secret

# OS artifacts
.DS_Store          # macOS
Thumbs.db          # Windows
desktop.ini        # Windows

# IDE
.vscode/settings.json
.idea/
```

Critical rules:

- `.venv/` **must** be in `.gitignore` before your first commit. If you accidentally commit the venv, it adds hundreds of MB to your repo history and will cause CI failures.
- `.env` **must** be in `.gitignore` . A `.env` file contains API keys and passwords. If it's ever committed, those credentials are compromised — even if you delete the file later, it remains in Git history.
- `*.csv` and similar data patterns prevent accidentally committing large data files. If you need a tiny sample for tests, commit it explicitly with `git add --force sample.csv` .

8.4 AGENTS.md

`AGENTS.md` is a repository-level guidance file defining rules, constraints, testing requirements, and scope boundaries for AI agents working in the codebase. It's the file that tells a CLI agent (or any human contributor) what the rules are for this repository.

Think of it as a CLAUDE.md or system prompt written for your project: it answers the questions any contributor needs answered before touching the code.

A minimal `AGENTS.md` for a data science project includes four sections:

1. **Testing requirements:** What must pass before any AI-assisted change is committed? (e.g., "All changes must pass `python test_environment.py` before committing.")
2. **Secrets policy:** What should never appear in a prompt or be committed? (e.g., "Do not include API keys, database passwords, or personal data in prompts.")
3. **Scope boundaries:** Which files/directories may the agent edit? Which are off-limits without human review? (e.g., "Agents may edit files in `src/` and `notebooks/` . Do not modify `requirements.txt` without human review.")
4. **Reproducibility standard:** What does "done" mean for an AI-assisted change? (e.g., "Local-first execution: the change must run locally and produce the expected output before it is committed.")

Creating an `AGENTS.md` is a writing exercise, not a tool-use exercise. You are not running an AI agent to complete this task — you are authoring the policy document that would govern such an agent. It requires no AI tool, paid or free, to write. Its purpose is to make explicit the rules a team would communicate to any contributor.

8.5 CHANGELOG.md

A **`CHANGELOG.md`** is a chronological record of notable changes to a project — what was added, changed, removed, or fixed, and when. It is standard in professional open-source and team repositories. Its purpose: a teammate should be able to read it and understand what changed between versions without reading every commit or PR.

Format convention: reverse-chronological, with dated entries:

```
# CHANGELOG

## [2026-03-15] — Add data validation pipeline
### Added
- `src/validate.py`: schema validation for admission records CSV
### Changed
- `requirements.txt`: added `pandera==0.17.3`
### Fixed
- Null handling in date parsing (was silently dropping rows)

## [2026-03-10] — Initial project setup
### Added
- Project scaffold: README, .gitignore, requirements.txt, setup.sh
- AGENTS.md: AI contribution policy
- test_environment.py: environment validation script
```

Label breaking changes explicitly so they can't be missed:

```
## [2026-04-01] — Migrate to psycopg3
### Breaking Changes / Migration Notes
- Replaced `psycopg2` with `psycopg3`. Run `pip install -r requirements.txt` to u
  Connection string syntax changed: use `conninfo` format instead of `dsn` keywor
```

8.6 Session Handoff Documents (HANDOFF.md)

In AI-assisted development workflows, it's common to maintain a **session handoff document** — a lightweight markdown file that carries rolling context between work sessions. Because AI agents do not retain memory between sessions, teams maintain a document describing:

- Current project state
- Last actions taken
- Next steps
- Open blockers

This is a form of asynchronous context management. If you've worked with a CLI agent on a multi-session project, you've likely encountered this pattern. The agent reads the handoff file at the start of each session to orient itself without re-reading the entire project history.

The format is project-specific, but a typical structure is: current status → recent actions → next actions → open questions. In this course, the `aispire-14005` system repo uses a `HANDOFF.md` for exactly this purpose — you'll see references to it in the course infrastructure.

8.7 setup.sh and Makefile

A **`setup.sh`** (or `Makefile`) automates the steps a new contributor needs to get a working environment. A good `setup.sh` is **idempotent**: running it twice produces the same result as running it once — it doesn't error if the venv already exists, it doesn't reinstall everything unnecessarily.

```
#!/usr/bin/env bash
set -euo pipefail

python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python test_environment.py
echo "Setup complete."
```

`set -euo pipefail` means: exit immediately if any command fails (`-e`), treat unset variables as errors (`-u`), and propagate pipe failures (`-o pipefail`). This prevents the script from silently continuing after a failure.

On Mac/Linux: After writing `setup.sh` , make it executable: `chmod +x setup.sh` . Then run it with `./setup.sh` .

On Windows: `setup.sh` won't run natively in CMD or PowerShell. Run it in Git Bash, or execute the commands manually.

8.8 Commit Messages as Documentation

A commit history is a log of decisions. Used well, it answers: "Why did this change? When did we break this? Who introduced this pattern?" Used poorly, it's noise.

The formula: **imperative subject line (50 chars) + blank line + body explaining the why.**

```
Fix null admission date crashing the pipeline

Upstream data sometimes sends empty strings instead of nulls.
pandas.to_datetime() raises ValueError on empty strings but
not on None. Added .replace("", None) before the datetime cast.
This does not affect rows with valid dates.
```

In this course, your commit messages are part of your PR and are read by autograders and TAs. Treat them as communication, not logging.

Environment Setup Troubleshooting

Use this section as a reference when something doesn't behave as expected during setup. More specific troubleshooting for each assignment is in the drill, lab, and integration guides.

`(.venv)` does not appear in my terminal prompt after activation The activation command did not run in the right shell, or it silently failed. Confirm:

- You ran the correct command for your OS and shell (see the table in Section 2.1)
- You are in the repo directory when you run it
- On Windows PowerShell: run `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser` once, then retry `Activate.ps1`

`ModuleNotFoundError` even though I ran `pip install -r requirements.txt` The install went to the system Python, not the venv. The venv was not active when you ran pip. Activate first (`(.venv)` prefix must be visible), then re-run `pip install -r requirements.txt` .

VS Code is using the wrong Python interpreter VS Code does not automatically detect your project venv. Press `Cmd+Shift+P` (Mac) or `Ctrl+Shift+P` (Windows/Linux) → type "Python: Select Interpreter" → choose the interpreter at `.venv/bin/python` (Mac/Linux) or `.venv\Scripts\python.exe` (Windows). If it does not appear, click "Enter interpreter path" and paste the path manually.

`pip install` appears to succeed but the package is still not found Check where pip installed the package: `pip show pandas` will print the install location. If it does not show `.venv` in the path, the venv was not active. Re-activate and re-run the install.

`python -m venv .venv` fails with a permissions error or "venv not found" On some systems, `python` resolves to Python 2 or is not in PATH. Try `python3 -m venv .venv` . If that fails, confirm Python 3.9+ is installed: `python3 --version` .

`pip install -r requirements.txt` fails on a package Check the error message: most failures are network issues (retry), version conflicts (check if you have an existing package installed globally), or a package that requires a system library. For this course, `pandas` and `matplotlib` should install cleanly in a fresh venv on any supported OS. If the install fails for these packages, post in the course channel with the full error output.

Key Terms

Term	Definition
`.gitignore`	A file specifying patterns of files Git should not track; prevents accidental commits of venvs, secrets, and data files
`AGENTS.md`	A repository-level guidance file defining rules, constraints, testing requirements, and scope boundaries for AI agents working in the codebase
`CHANGELOG.md`	A human-readable file at the repo root documenting notable changes over time: what was added, changed, removed, or fixed, and when
`session handoff document`	A lightweight markdown file (e.g., `HANDOFF.md`) maintained in AI-assisted workflows to carry rolling context between work sessions; records current state, recent actions, and next steps
`agentic IDE`	A development environment with an embedded AI agent that can propose multi-file edits and execute tasks
`AI coding agent`	An AI system that can read, modify, and generate code across multiple files using structured instructions
`branch`	A movable pointer to a commit; a named line of development
`CLI agent`	An AI coding agent operated from the command line that can inspect, edit, and refactor repositories
`conda`	Cross-platform package and environment manager that supports non-Python packages (e.g., CUDA, R libraries); conceptual reference in this course
`conflict markers`	`<<<<<<<` , `=======` , `>>>>>>>` — three lines inserted by Git to delimit the two conflicting versions of a file region during a merge conflict
`container`	A lightweight, isolated process with its own filesystem, sharing the host OS kernel; faster and lighter than a virtual machine
`context window`	The amount of text or code an AI model can consider at one time when generating output
`continuous integration (CI)`	Automated workflows that run tests and checks on each push or pull request; ensures code quality before merging
`deterministic`	Code or a process that always produces the same result given the same input; contrast with non-deterministic
`Docker image`	A read-only snapshot built from a Dockerfile; used to create containers
`Dockerfile`	A script of instructions for building a Docker image
`environment.yml`	YAML file specifying a conda environment's name, channels, and dependencies
`evaluation loop`	A structured process where an agent reviews and tests its own output before finalizing changes; also called a self-check or critique step
`function calling`	A model capability that allows the AI to make structured requests to predefined tools or APIs as part of generating a response
`hallucination`	When an AI model generates incorrect or fabricated information — such as a non-existent function or wrong API signature — with confidence
`HEAD`	Git's pointer to the currently checked-out commit or branch
`human-in-the-loop (HITL)`	A development approach where humans review and approve AI-generated outputs before deployment or commit
`idempotent script`	A script that can be run multiple times without changing the result beyond the initial application
`kernel (Jupyter)`	The computational engine that executes notebook code; tied to a specific Python environment
`local-first execution`	Running code and verifying outputs locally before pushing changes to a remote repository
`magic command`	Jupyter line or cell commands prefixed with `%` or `%%` that control notebook behavior; processed by the IPython kernel, not Python
`merge conflict`	Arises when two branches modify the same lines of the same file and Git cannot auto-resolve the difference; requires manual resolution
`planning phase`	An initial reasoning step where an AI agent outlines intended changes before modifying code
`pre-commit hook`	A script that runs automatically before a commit is finalized to enforce quality checks; blocks the commit if it exits non-zero
`prompt injection`	A malicious attempt to manipulate an AI system by inserting unintended instructions into its input
`pull request (PR)`	A proposal to merge one branch into another, with a code review and discussion interface
`remote`	A reference to a Git repository hosted elsewhere (e.g., on GitHub)
`requirements.txt`	A text file listing pip package dependencies with optional version pins; the environment contract for a Python project
`scaffold`	A starter structure or template used to accelerate implementation
`secrets management`	Practices for preventing credentials and sensitive data from being exposed in code, commits, or prompts
`spec-first development`	Defining requirements and acceptance criteria before implementation begins
`system prompt`	The persistent instructions that define an AI agent's role, constraints, and behavior at the start of a session
`tool invocation`	The structured calling of external tools or commands by an AI agent during execution
`verification-first workflow`	A practice of testing and validating outputs before accepting or merging changes — applies to AI-generated code and your own
`venv`	Python's built-in virtual environment module; creates an isolated Python installation with its own packages
`virtual environment`	An isolated Python installation with its own packages, separate from the system Python and other projects

---

Summary

Module 1 establishes the engineering foundation every subsequent module depends on. The core insight is that professional data science work requires reproducibility at every layer: your Python environment must be specified so teammates can recreate it exactly, your code must live in a branch so it can be reviewed before it affects others, and your repository must be structured so any contributor — human or AI agent — can orient themselves without asking questions. AI coding tools amplify these skills; they don't substitute for them. A tool that generates a `requirements.txt` you don't understand, or a PR description you didn't write, isn't helping you build the foundation — it's building on sand. Master the environment and Git workflow first; the tools become genuinely useful after that.

What's Next

- **Tonight (after the concept session):** Core Skills Drill — the link activates in TalentLMS Unit 3 after the concept session. You will submit a GitHub PR URL — the autograder verifies your work automatically.
- **Days 2–3:** Applied Lab (`lab-1-git-workflows`) — configure a version-controlled Python project environment for a public health data analysis scenario.
- **Days 4–5:** Integration Task (`integration-1-dev-environment`) — extend your lab repo into a complete, team-ready collaborative project with a professional README, `setup.sh` , and `AGENTS.md` .
- **Concept session tonight:** The concept session builds schema around the *why* — why environments are engineering contracts, why branches are coordination protocols. Come with this reading complete.

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.